



VAT MANUAL

Xuan, Hao
xuan13hao@ku.edu

Contents

1. Overview	4
2. Requirements.....	5
2.1. Supported Operating Systems.....	5
2.2. Required Libraries	5
2.3. System Requirements	5
2.3.1. Hardware Requirements	5
2.3.2. Memory Usage.....	5
2.3.3. Disk Usage.....	5
2.3.4. Runtime Considerations	6
2.3.5. Checking AVX2 Support.....	6
2.3.6. Note	6
3. Quick Start	7
3.1. Building a VAT Database.....	7
3.2. Running Alignment	7
3.2.1. NGS Whole-genome sequencing mapping.....	7
3.2.2. TGS Whole-genome sequencing mapping.....	7
3.2.3. NGS ChIP-seq mapping.....	7
3.2.4. TGS ChIP-seq mapping	7
3.2.5. NGS ATAC-seq mapping	7
3.2.6. TGS ATAC-seq mapping.....	7
3.2.7. NGS 16S rRNA mapping.....	8
3.2.8. TGS 16S rRNA mapping	8
3.2.9. NGS Metagenomic mapping	8
3.2.10. TGS Metagenomic mapping.....	8
3.2.11. TGS HiFi mapping – experimental mode.....	8
3.2.12. NGS RNA-seq mapping	8
3.2.13. TGS RNA-seq mapping.....	8
3.2.14. NGS Circular RNA mapping	8
3.2.15. TGS Circular RNA mapping	8
3.2.16. NGS CLASH mapping.....	8
3.2.17. TGS CLASH mapping	9
3.2.18. NGS Hi-C mapping	9
3.2.19. TGS Hi-C mapping.....	9
3.2.20. DNA homology search	9

3.2.21. Whole genome alignment.....	9
3.2.22. Protein homology search – fast mode	9
3.2.23. Protein homology search – sensitive mode	9
3.2.24. Translated search – fast mode	9
3.2.25. Translated search – sensitive mode	9
3.3. View Archive Files	9
3.4. Supported Alignment Tasks.....	10
4. Input.....	11
4.1. Reference Input.....	11
4.2. Query Input.....	11
4.3. Note.....	11
5. Output	12
5.1. SAM Format	12
5.2. PAF Format	12
5.3. Tabular Format.....	12
5.4. Pairwise Format.....	13
5.5. Field Naming Convention	13
6. Interface Hierarchy Architecture	14
6.1. Hierarchical Structure	14
6.2. Tier Definitions.....	15
6.2.1. Tier 1.....	15
6.2.2. Tier 2.....	15
6.2.3. Tier 3.....	15
6.2.4. Tier 4.....	15
6.3. Command Composition	19
7. Alignment Task–Specific Parameters.....	20
7.1. Whole-genome Sequencing Mapping	20
7.1.1. Next-generation sequencing reads mapping	20
7.1.2. Third-generation sequencing reads mapping	20
7.2. ChIP-seq Mapping	20
7.2.1. Next-generation sequencing reads mapping	20
7.2.2. Third-generation sequencing reads mapping	20
7.3. ATAC-seq Mapping	21
7.3.1. Next-generation sequencing reads mapping	21
7.3.2. Third-generation sequencing reads mapping	21
7.4. 16S rRNA Sequencing Mapping	21

7.4.1. Next-generation sequencing reads mapping	21
7.4.2. Third-generation sequencing reads mapping	21
7.5. Metagenomic Mapping	22
7.5.1. Next-generation sequencing reads mapping	22
7.5.2. Third-generation sequencing reads mapping	22
7.6. RNA-seq Mapping	22
7.6.1. Next-generation sequencing reads mapping	22
7.6.2. Third-generation sequencing reads mapping	22
7.7. CircRNA Mapping.....	23
7.7.1. Next-generation sequencing reads mapping	23
7.7.2. Third-generation sequencing reads mapping	23
7.8. CLASH Mapping.....	24
7.8.1. Next-generation sequencing reads mapping	24
7.8.2. Third-generation sequencing reads mapping	24
7.9. Hi-C Mapping	25
7.9.1. Next-generation sequencing reads mapping	25
7.9.2. Third-generation sequencing reads mapping	25
7.10. DNA Homology Search.....	25
7.11. Whole Genome Alignment.....	26
7.12. Protein Homology Search	26
7.12.1. Fast.....	26
7.12.2. Sensitive	26
7.13. Translated Homology Search	26
7.13.1. Fast.....	26
7.13.2. Sensitive	27

1. Overview

VAT (Versatile Alignment Tool) is a high-performance sequence alignment framework for both nucleotide and protein sequences. It provides a unified command-line interface that supports a wide range of biological applications through a hierarchical design. VAT supports multiple alignment paradigms, including:

- Whole-genome sequencing (WGS) read mapping
- Chromatin accessibility and protein–DNA interaction mapping (ATAC-seq, ChIP-seq)
- Metagenomic read mapping and microbial profiling
- RNA-seq read mapping
- Circular RNA (circRNA) detection
- Chimeric and fusion read mapping
- Chromatin conformation capture (Hi-C) read mapping
- DNA homology search
- Whole-genome alignment
- Protein homology search
- Translated nucleotide-to-protein alignment

VAT is optimized for high performance through efficient indexing, flexible seeding strategies, and SIMD-based acceleration, enabling scalable analysis from small genomes to large-scale sequencing datasets.

2. Requirements

2.1. Supported Operating Systems

VAT is designed for Unix-like environments and has been primarily tested on:

- 64-bit Linux (x86_64)

Support for other operating systems (e.g., macOS) is not officially guaranteed and may require additional configuration steps.

VAT also supports containerized execution via Singularity/Apptainer using .sif images, enabling reproducible and portable deployments across different computing environments. This enables VAT to be deployed in high-performance computing (HPC) environments without manual dependency management.

2.2. Required Libraries

VAT is implemented as a standalone command-line tool and does not require external runtime dependencies beyond standard system libraries.

Requirements include:

- C/C++ standard runtime environment
- POSIX-compatible system
- AVX2 instruction support (required)

Optional (depending on compilation settings):

- OpenMP (for multithreading support)

2.3. System Requirements

2.3.1. Hardware Requirements

- 64-bit processor
- AVX2 instruction set support (required)
- Multi-core CPU recommended
- Intel CPUs: Haswell architecture and newer
- AMD CPUs: Excavator architecture and newer

2.3.2. Memory Usage

Memory usage depends on:

- Reference database size
- Query dataset size
- Alignment task and mode

Typical estimates:

- Small genomes (e.g., bacteria): ~1–4 GB RAM
- Human genome-scale reference: ~8–32 GB RAM

2.3.3. Disk Usage

Disk usage includes:

- VAT database files
- Alignment output files (SAM, PAF, tabular)
- Optional archive files (.vatr)

Storage requirements scale approximately with:

- Reference database size
- Number of reported alignments

2.3.4. Runtime Considerations

Runtime depends on:

- Query size (number and length of sequences)
- Reference database size
- Alignment mode (e.g., short-read vs. WGA vs. homology)
- Number of CPU threads (-p)

2.3.5. Checking AVX2 Support

On Linux systems:

```
grep -o 'avx2' /proc/cpuinfo | head -1
```

or:

```
lscpu | grep -i avx2
```

If the output contains avx2, the system supports AVX2 instructions.

2.3.6. Note

VAT will not run on systems that do not support AVX2. When using VAT via .sif containers, the host system must still support AVX2, as containerization does not emulate CPU instruction sets.

3. Quick Start

The VAT workflow mainly consists of two main steps: database construction and sequence alignment.

3.1. Building a VAT Database

In the first step, reference sequences are provided in FASTA format and used to construct a VAT database for efficient indexing and alignment.

Usage:

```
VAT makevatdb --in <reference.fa> --dbtype nucl -p <threads>
VAT makevatdb --in <reference.fa> --dbtype prot -p <threads>
```

3.2. Running Alignment

In the second step, query sequences are aligned against the pre-built VAT database across major application domains: Next-generation sequencing reads (NGS) mapping, Third-generation sequencing reads (TGS) mapping, homology search, and whole-genome alignment. Query sequences can be provided in FASTA or FASTQ format.

Usage:

```
VAT <Tier1> <Tier2> <Tier3> <Tier4>
```

3.2.1. NGS Whole-genome sequencing mapping

Usage:

```
VAT nucl short WGS -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.2. TGS Whole-genome sequencing mapping

Usage:

```
VAT nucl long WGS -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.3. NGS ChIP-seq mapping

Usage:

```
VAT nucl short ChIPseq -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.4. TGS ChIP-seq mapping

Usage:

```
VAT nucl long ChIPseq -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.5. NGS ATAC-seq mapping

Usage:

```
VAT nucl short ATACseq -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.6. TGS ATAC-seq mapping

Usage:


```
VAT nucl long ATACseq -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.7. NGS 16S rRNA mapping

Usage:

```
VAT nucl short 16S -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.8. TGS 16S rRNA mapping

Usage:

```
VAT nucl long 16S -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.9. NGS Metagenomic mapping

Usage:

```
VAT nucl short MG -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.10. TGS Metagenomic mapping

Usage:

```
VAT nucl long MG -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.11. TGS HiFi mapping – experimental mode

Usage:

```
VAT nucl long HiFi -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.12. NGS RNA-seq mapping

Usage:

```
VAT nucl short RNAseq -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.13. TGS RNA-seq mapping

Usage:

```
VAT nucl long RNAseq -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.14. NGS Circular RNA mapping

Usage:

```
VAT nucl short CircRNA -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.15. TGS Circular RNA mapping

Usage:

```
VAT nucl long CircRNA -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.16. NGS CLASH mapping

Usage:

```
VAT nucl short CLASH -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.17. TGS CLASH mapping

Usage:

```
VAT nucl long CLASH -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.18. NGS Hi-C mapping

Usage:

```
VAT nucl short HiC -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.19. TGS Hi-C mapping

Usage:

```
VAT nucl long HiC -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

3.2.20. DNA homology search

Usage:

```
VAT nucl homology null -d <ref.fa> -q <query.fa> -o <out.tab> -f tab
```

3.2.21. Whole genome alignment

Usage:

```
VAT nucl wga null -d <genome.fa> -q <query.fa> -a <output>
```

Post-processing:

```
VAT view -a <output> -o <output.tab> -f tab
```

3.2.22. Protein homology search – fast mode

Usage:

```
VAT protein homology fast -d <db.fa> -q <query.fa> -o <out.tab> -f tab
```

3.2.23. Protein homology search – sensitive mode

Usage:

```
VAT protein homology sensitive -d <db.fa> -q <query.fa> -o <out.tab> -f tab
```

3.2.24. Translated search – fast mode

Usage:

```
VAT blastx homology fast -d <db.fa> -q <dna_reads.fa> -o <out.tab> -f tab
```

3.2.25. Translated search – sensitive mode

Usage:

```
VAT blastx homology sensitive -d <db.fa> -q <dna_reads.fa> -o <out.tab> -f tab
```

3.3. View Archive Files

View or convert VAT alignment archive files (.vatr) into standard output formats. This command is typically used after genome-scale alignment tasks such as [nucl](#) and [wga](#) to enable downstream analysis.

Usage:

```
VAT view -a <input> -o <output> -f <outfmt> -p <threads>
```

3.4. Supported Alignment Tasks

VAT uses a hierarchical command-line design to standardize alignment workflows in Section 6. VAT organizes alignment tasks using a hierarchical command structure (see Table 1). Task-specific parameters for each alignment mode are described in Section 7.

Table 1: Supported alignment tasks of VAT. Tier 1 specifies the primary function (e.g., **nucl**, **protein**, **blastx**), Tier 2 defines the alignment strategy (e.g., **short**, **long**, **wga**, **homology**), and Tier 3 indicates the application-specific mode (e.g., **WGS**, **RNAseq**, **MG**) or sensitivity level (**fast**, **sensitive**). For tasks that do not require a Tier 3 specification (e.g., whole-genome alignment (**wga**) and homology search (**homology**) under **nucl**), the Tier 3 field must be explicitly set to "null".

Tier1	Tier2	Tier3	Alignment tasks
nucl	short	WGS	Whole-Genome Sequencing Mapping
nucl	short	ChIPseq	ChIP-seq Mapping
nucl	short	ATACseq	ATAC-seq Mapping
nucl	short	MG	Metagenomic Mapping
nucl	short	16S	16S rRNA Mapping
nucl	short	RNAseq	RNA-seq Mapping
nucl	short	CircRNA	Circular RNA Mapping
nucl	short	CLASH	CLASH Mapping
nucl	short	HiC	Hi-C Mapping
nucl	long	WGS	Whole-Genome Sequencing Mapping
nucl	long	ChIPseq	ChIP-seq Mapping
nucl	long	ATACseq	ATAC-seq Mapping
nucl	long	MG	Metagenomic Mapping
nucl	long	16S	16S rRNA Mapping
nucl	long	RNAseq	RNA-seq Mapping
nucl	long	CircRNA	Circular RNA Mapping
nucl	long	CLASH	CLASH Mapping
nucl	long	HiC	Hi-C Mapping
nucl	long	HiFi	HiFi Mapping
nucl	wga	null	Whole Genome Alignment
nucl	homology	null	DNA Homology Search
protein	homology	fast	Protein Homology Search
protein	homology	sensitive	Protein Homology Search
blastx	homology	fast	Translated Search
blastx	homology	sensitive	Translated Search

4. Input

VAT accepts standard biological sequence formats for both reference and query inputs. Supported formats depend on the command and application mode.

4.1. Reference Input

Reference sequences are used to build VAT databases and must be provided in FASTA format. Supported format:

- FASTA (.fa, .fasta)
- Gzipped FASTA (.fa.gz, .fasta.gz)

Requirements:

- Each sequence must have a header line starting with >
- Sequence identifiers should be unique
- Multi-line sequences are supported

4.2. Query Input

Query sequences are used for alignment and can be provided in either FASTA or FASTQ format. Supported formats:

- FASTA (.fa, .fasta)
- FASTQ (.fq, .fastq)
- Gzipped versions supported (.gz)

4.3. Note

- Input files may be plain text or gzipped
- File format is automatically detected based on content
- Large input files are supported; performance depends on system resources

5. Output

VAT supports multiple output formats for alignment results. The output format can be specified using the **-f** or **--outfmt** option.

5.1. SAM Format

The SAM (Sequence Alignment/Map) format is widely used for representing sequence alignments and is compatible with many downstream analysis tools. Each alignment line contains 11 mandatory columns followed by optional tags.

- **QNAME** – Query sequence name
- **FLAG** – Bitwise alignment flag indicating strand and mapping status
- **RNAME** – Reference sequence name
- **POS** – 1-based leftmost mapping position on the reference
- **MAPQ** – Mapping quality score
- **CIGAR** – CIGAR string describing alignment operations
- **RNEXT** – Reference name of the next read in the template (* for single-end reads)
- **PNEXT** – Position of the next read in the template (0 for single-end reads)
- **TLEN** – Template length (0 for single-end reads)
- **SEQ** – Query sequence
- **QUAL** – Base quality string (* if not available)

For simple alignments, VAT typically sets the FLAG value to:

- 0 – forward strand alignment
- 16 – reverse strand alignment

5.2. PAF Format

VAT also supports output in PAF (Pairwise mApping Format), used for the whole genome alignment mode.

- **query_name** – Query sequence name
- **query_length** – Query length
- **query_start** – Query start (0-based)
- **query_end** – Query end
- **strand** – Alignment strand (+ or -)
- **target_name** – Target (reference) sequence name
- **target_length** – Target length
- **target_start** – Target start
- **target_end** – Target end
- **match_bases** – Number of matching bases in the alignment
- **MAPQ** – Mapping quality score (0–60)

5.3. Tabular Format

VAT also supports a BLAST-like tabular output format, which is compact and easy to parse for downstream analysis.

- **query_name** – Query sequence identifier (read name)
- **target_name** – Subject sequence identifier (reference or database sequence name)

- **identity** – Percentage identity within the aligned region, ranging from 0 to 100
- **alignment_length** – Total alignment length, including matches, mismatches, and gaps
- **mismatch** – Number of mismatching bases or amino acids in the alignment
- **gap_open** – Number of gap openings (insertion or deletion segments)
- **query_start** – Alignment start position on the query sequence (1-based coordinate)
- **query_end** – Alignment end position on the query sequence (1-based coordinate)
- **target_start** – Alignment start position on the subject (reference) sequence (1-based coordinate)
- **target_end** – Alignment end position on the subject sequence (1-based coordinate)
- **evalue** – Expectation value (E-value) of the alignment. Smaller values indicate more significant matches
- **bitscore** – Bit score of the alignment. Larger values indicate higher alignment quality

5.4. Pairwise Format

VAT also supports a BLAST-like pairwise output format, used for the whole genome alignment mode.

- **query_name** – Query sequence identifier (read name)
- **target_name** – Subject sequence identifier (reference or database sequence name), together with the total sequence length
- **score** – Alignment score reported as a bit score and the corresponding raw alignment score
- **evalue** – Expectation value (E-value) of the alignment. Smaller values indicate more significant matches
- **identities** – Number and percentage of identical residues within the aligned region
- **positives** – Number and percentage of similar residues (reported for protein alignments)
- **gaps** – Number and percentage of gap openings in the alignment
- **frame** – Translation frame of the query sequence for translated alignments (reported in blastx mode)
- **query alignment** – Displays the aligned query sequence segment together with its start and end coordinates (1-based)
- **match line** – A visual representation of matches and mismatches between the query and subject sequences. Identical residues are typically indicated by symbols such as |
- **subject alignment** – Displays the aligned subject sequence segment together with its start and end coordinates (1-based)
- **Line width** – Alignments are printed in segments of 60 characters per line, following the traditional BLAST pairwise layout

5.5. Field Naming Convention

VAT adopts a consistent field naming scheme across output formats. Fields are normalized to a unified schema (e.g., `query_name`, `target_name`, `identity`), while preserving compatibility with standard formats such as SAM and PAF.

6. Interface Hierarchy Architecture

6.1. Hierarchical Structure

VAT employs a hierarchical command-line architecture to unify diverse sequence alignment tasks within a single framework. Each command is organized into multiple tiers, with each tier representing a distinct semantic level of the operation. At the highest level, Tier 1 defines the primary function, such as database construction, nucleotide alignment, protein alignment, translated search, or archive processing. Tier 2 specifies the alignment type, including short-read mapping, long-read mapping, whole-genome alignment, and homology search. Tier 3 refines the command by indicating either the biological application scenario or the sensitivity mode. For tasks that do not require an application-specific submodule, this tier is set to **null**. Finally, Tier 4 consists of execution options, such as input/output files, output format, thread number, and alignment parameters. The general command syntax is illustrated in Figure 1:

```
VAT <Tier1> <Tier2> <Tier3> <Tier4>
```

This design improves usability, consistency, and extensibility by allowing different applications to share a common command grammar while preserving task-specific flexibility.

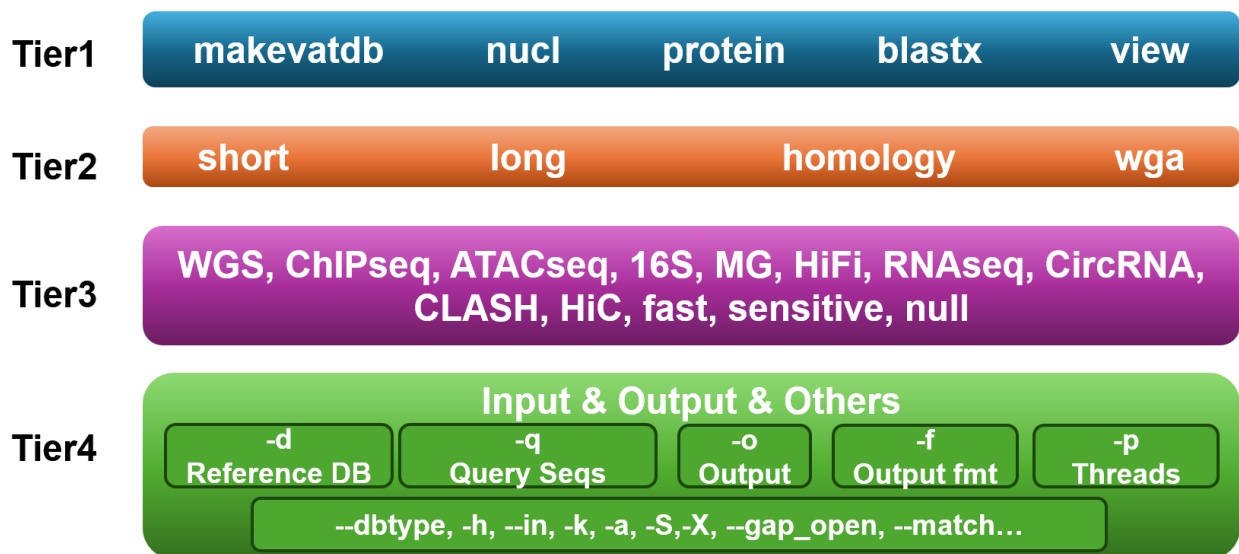


Figure 1: Overview of the VAT hierarchical command structure. Commands are composed of four tiers representing function, alignment type, application mode or sensitivity, and execution options.

6.2. Tier Definitions

6.2.1. Tier 1

Defines the primary operation of VAT:

- **makevatdb** — database construction
- **nucl** — nucleotide alignment
- **protein** — protein alignment
- **blastx** — translated nucleotide-to-protein alignment
- **view** — archive processing

6.2.2. Tier 2

Defines the alignment or analysis type:

- **short** — short-read mapping, NGS reads (e.g., Illumina)
- **long** — long-read mapping, TGS reads (e.g., ONT, PacBio)
- **wga** — whole-genome alignment
- **homology** — sequence homology search

6.2.3. Tier 3

Defines the application scenario:

Nucleotide modes:

- **WGS, RNAseq, CircRNA, MG, CLASH, HiC, 16S, ChIPseq, ATACseq, HiFi, null**

Protein and blastx modes:

- **fast, sensitive**

6.2.4. Tier 4

Database construction:

--in

Type: FASTA file

Default: none

Input reference file (plain or gzipped).

--dbtype

Type: nucl | prot

Default: nucl

Specifies the database type:

nucl – nucleotide database

prot – protein database

-p, --threads

Type: integer

Default: 4

Number of CPU threads used during database construction.

Running alignment:

--report_id

Type: float (%)

Default: 0

Minimum percentage identity required to report an alignment.

-e, --evaluate

Type: float

Default: 0.001

Maximum E-value threshold for reporting alignments.

-k, --maxtarget_seqs

Type: integer

Default: 25

Maximum number of target sequences to report per query.

-S

Type: integer

Default: 15 (DNA), 8 (protein)

Length of the seed (k-mer) used for initial sequence matching.

Smaller values increase sensitivity but may reduce alignment speed.

-w

Type: integer

Default: 0

Window size for search.

-d, --db

Type: file

Path to the VAT reference database file.

-p, --threads

Type: integer

Default: 4

Number of CPU threads used during alignment.

-q, --query

Type: file

Input query sequence file.

Supported formats typically include FASTA and FASTQ (optionally gzipped).

-f, --outfmt

Type: tab | sam | paf | pairwise

Default: tab

Specifies the output format:

tab — tabular alignment output

sam — SAM format (compatible with downstream tools)

paf — Pairwise mApping Format

pairwise — human-readable pairwise alignment format

-o, --out

Type: file

Default: none

Output file for converted alignment results

--gapopen

Type: integer

Default: -2

Penalty for opening a new gap.

--gapextend

Type: integer

Default: -1

Penalty for extending an existing gap during alignment.

--match

Type: integer / matrix

Default: +5 (DNA), BLOSUM62 (protein)

Score assigned to matching bases or amino acids.

--mismatch

Type: integer / matrix

Default: -3 (DNA), BLOSUM62 (protein)

Penalty assigned to mismatched bases or amino acids.

--xdrop

Type: integer

Default: 18

X-drop threshold for ungapped extension.

Controls early termination of extension when score drops.

-X, --xdrop_gapped

Type: integer

Default: 18

X-drop threshold for gapped extension.

--max_seed_gap_query

Type: integer

Default: 150

Maximum allowed distance between two seeds on the query sequence.

--max_seed_gap_ref

Type: integer

Default: 150

Maximum allowed distance between two seeds on the reference sequence.

--seed_pattern_upstream, --seed_upstream_score

Type: string + score

Default: None (sequence), 0 (score)

Sequence pattern expected upstream of seeds and its associated bonus/penalty score.

--seed_pattern_downstream, --seed_downstream_score

Type: string + score

Default: None (sequence), 0 (score)

Sequence pattern expected downstream of seeds and its associated bonus/penalty score.

--interchr

Type: boolean

Default: false

Allow chaining of seeds across different chromosomes.

-N

Type: integer

Default: 1

Number of spaced seed patterns used during alignment.

Increasing this may improve sensitivity at the cost of speed.

--ra

Type: string

Default: null

Reduced amino acid alphabet used for protein alignment.

Helps improve sensitivity by grouping similar amino acids.

Viewing archive files:

-a, --vaa

Type: file (optional)

Default: none

Archive file used to store VAT alignment results in the internal .vatr format for the whole genome alignment part. This archive can later be inspected or converted using the vat view command.

-o, --out

Type: file

Default: none

Output file for converted alignment results

-f, --outfmt

Type: tab | sam | paf | pairwise

Default: tab

Specifies the output format:

tab — tabular alignment output

sam — SAM format (compatible with downstream tools)

paf — Pairwise mApping Format

pairwise — human-readable pairwise alignment format

-p, --threads

Type: integer

Default: 4

Number of CPU threads used during archive processing

6.3. Command Composition

The four-tier command structure (Tier1–Tier4) applies to alignment commands ([nucl](#), [protein](#), [blastx](#)). Non-alignment commands, such as [makevatdb](#) and [view](#), do not follow the tiered structure.

```
VAT makevatdb <Tier4>
```

```
VAT nucl short <Tier3> <Tier4>
```

```
VAT nucl long <Tier3> <Tier4>
```

```
VAT nucl homology null <Tier4>
```

```
VAT nucl wga null <Tier4>
```

```
VAT protein homology <Tier3> <Tier4>
```

```
VAT blastx homology <Tier3> <Tier4>
```

```
VAT view <Tier4>
```

7. Alignment Task–Specific Parameters

7.1. Whole-genome Sequencing Mapping

7.1.1. Next-generation sequencing reads mapping

Usage:

```
VAT nucl short WGS -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 15 → **-S** 15
- Minimizer window size: 5 → **-w** 5
- Match: +5 → **--match** 5
- Mismatch: -2 → **--mismatch** -2

7.1.2. Third-generation sequencing reads mapping

Usage:

```
VAT nucl long WGS -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 15 → **-S** 15
- Minimizer window size: 10 → **-w** 10
- Match: +2 → **--match** 2
- Mismatch: -4 → **--mismatch** -4
- Gap open: -8 → **--gapopen** -8

7.2. ChIP-seq Mapping

7.2.1. Next-generation sequencing reads mapping

Usage:

```
VAT nucl short ChIP-seq -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 15 → **-S** 15
- Minimizer window size: 5 → **-w** 5
- Match: +5 → **--match** 5
- Mismatch: -2 → **--mismatch** -2

7.2.2. Third-generation sequencing reads mapping

Usage:

```
VAT nucl long ChIP-seq -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 15 → **-S** 15
- Minimizer window size: 10 → **-w** 10
- Match: +2 → **--match** 2
- Mismatch: -4 → **--mismatch** -4

- Gap open: -8 → **--gapopen** -8

7.3. ATAC-seq Mapping

7.3.1. Next-generation sequencing reads mapping

Usage:

```
VAT nucl short ATAC-seq -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 15 → **-S** 15
- Minimizer window size: 5 → **-w** 5
- Match: +5 → **--match** 5
- Mismatch: -2 → **--mismatch** -2

7.3.2. Third-generation sequencing reads mapping

Usage:

```
VAT nucl long ATAC-seq -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 15 → **-S** 15
- Minimizer window size: 10 → **-w** 10
- Match: +2 → **--match** 2
- Mismatch: -4 → **--mismatch** -4
- Gap open: -8 → **--gapopen** -8

7.4. 16S rRNA Sequencing Mapping

7.4.1. Next-generation sequencing reads mapping

Usage:

```
VAT nucl short 16S -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 15 → **-S** 15
- Minimizer window size: 5 → **-w** 5
- Match: +5 → **--match** 5
- Mismatch: -2 → **--mismatch** -2

7.4.2. Third-generation sequencing reads mapping

Usage:

```
VAT nucl long 16S -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 15 → **-S** 15
- Minimizer window size: 10 → **-w** 10
- Match: +2 → **--match** 2
- Mismatch: -4 → **--mismatch** -4

- Gap open: -8 → **--gapopen** -8

7.5. Metagenomic Mapping

7.5.1. Next-generation sequencing reads mapping

Usage:

```
VAT nucl short MG -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 15 → **-S** 15
- Minimizer window size: 5 → **-w** 5
- Match: +5 → **--match** 5
- Mismatch: -2 → **--mismatch** -2

7.5.2. Third-generation sequencing reads mapping

Usage:

```
VAT nucl long MG -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 15 → **-S** 15
- Minimizer window size: 10 → **-w** 10
- Match: +2 → **--match** 2
- Mismatch: -4 → **--mismatch** -4
- Gap open: -8 → **--gapopen** -8

7.6. RNA-seq Mapping

7.6.1. Next-generation sequencing reads mapping

Usage:

```
VAT nucl short RNA-seq -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 14 → **-S** 14
- Minimizer window size: 5 → **-w** 5
- Match: +1 → **--match** 1
- Mismatch: -2 → **--mismatch** -2
- X-drop: 15 → **--xdrop** 15
- X-drop (gapped): 15 → **-X** 15
- Seed gap (query): 1000 → **--max_seed_gap_query** 1000
- Seed gap (ref): 200000 → **--max_seed_gap_ref** 200000
- Splice donor: GT
→ **--seed_pattern_upstream** GT **--seed_upstream_score** 4
- Splice acceptor: AG
→ **--seed_pattern_downstream** AG **--seed_downstream_score** 4

7.6.2. Third-generation sequencing reads mapping

Usage:

```
VAT nucl long RNA-seq -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 15 → **-S** 15
- Minimizer window size: 5 → **-w** 5
- Match: +1 → **--match** 1
- Mismatch: -2 → **--mismatch** -2
- Gap extend: -1 → **--gapextend** -1
- Gap open: -2 → **--gapopen** -2
- X-drop: 20 → **--xdrop** 20
- X-drop (gapped): 20 → **-X** 20
- Seed gap (query): 2000 → **--max_seed_gap_query** 2000
- Seed gap (ref): 200000 → **--max_seed_gap_ref** 200000
- Splice donor: GT (+4)
→ **--seed_pattern_upstream** GT **--seed_upstream_score** 4
- Splice acceptor: AG (+4)
→ **--seed_pattern_downstream** AG **--seed_downstream_score** 4

7.7. CircRNA Mapping

7.7.1. Next-generation sequencing reads mapping

Usage:

```
VAT nucl short CircRNA -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 14 → **-S** 14
- Minimizer window size: 2 → **-w** 2
- Match: +1 → **--match** 1
- Mismatch: -3 → **--mismatch** -3
- Gap extend: -2 → **--gapextend** -2
- Gap open: -5 → **--gapopen** -5
- X-drop: 15 → **--xdrop** 15
- X-drop (gapped): 15 → **-X** 15
- Seed gap (query): 1000 → **--max_seed_gap_query** 1000
- Seed gap (ref): 200000 → **--max_seed_gap_ref** 200000
- Back-splice signals:
 - Upstream: →
--seed_pattern_upstream GT **--seed_upstream_score** 4
 - Downstream: →
--seed_pattern_downstream AG **--seed_downstream_score** 4

7.7.2. Third-generation sequencing reads mapping

Usage:

```
VAT nucl long CircRNA -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 14 → **-S** 14

- Minimizer window size: 10 → **-w** 10
- Match: +1 → **--match** 1
- Mismatch: -3 → **--mismatch** -3
- Gap extend: -2 → **--gapextend** -2
- Gap open: -5 → **--gapopen** -5
- X-drop: 15 → **--xdrop** 15
- X-drop (gapped): 15 → **-X** 15
- Seed gap (query): 1000 → **--max_seed_gap_query** 1000
- Seed gap (ref): 200000 → **--max_seed_gap_ref** 200000
- Back-splice signals:
 - Upstream: →
--seed_pattern_upstream GT **--seed_upstream_score** 4
 - Downstream: →
--seed_pattern_downstream AG **--seed_downstream_score** 4

7.8. CLASH Mapping

7.8.1. Next-generation sequencing reads mapping

Usage:

```
VAT nucl short CLASH -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 11 → **-S** 11
- Minimizer window size: 1 → **-w** 1
- Match: +5 → **--match** 5
- Mismatch: -4 → **--mismatch** -4
- Gap extend: -3 → **--gapextend** -3
- Gap open: -5 → **--gapopen** -5
- X-drop: 10 → **--xdrop** 10
- X-drop gapped: 10 → **-X** 10
- Inter-chromosome: enabled → **--interchr**

7.8.2. Third-generation sequencing reads mapping

Usage:

```
VAT nucl long CLASH -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 11 → **-S** 11
- Minimizer window size: 5 → **-w** 5
- Match: +5 → **--match** 5
- Mismatch: -4 → **--mismatch** -4
- Gap extend: -3 → **--gapextend** -3
- Gap open: -5 → **--gapopen** -5
- X-drop: 10 → **--xdrop** 10
- X-drop gapped: 10 → **-X** 10
- Inter-chromosome: enabled → **--interchr**

7.9. Hi-C Mapping

7.9.1. Next-generation sequencing reads mapping

Usage:

```
VAT nucl short Hi-C -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 13 → **-S** 13
- Minimizer window size: 5 → **-w** 5
- Match: +1 → **--match** 1
- Mismatch: -3 → **--mismatch** -3
- Gap extend: -1 → **--gapextend** -1
- Gap open: -6 → **--gapopen** -6
- Seed gap (query): 5000 → **--max_seed_gap_query** 5000
- Seed gap (ref): 5000 → **--max_seed_gap_ref** 5000
- Restriction sites:
 - Upstream: →
--seed_pattern_upstream AAGCTT **--seed_upstream_score** 4
 - Downstream: →
--seed_pattern_downstream GATC **--seed_downstream_score** 4
- Inter-chromosome: enabled → **--interchr**

7.9.2. Third-generation sequencing reads mapping

Usage:

```
VAT nucl long Hi-C -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 13 → **-S** 13
- Minimizer window size: 10 → **-w** 10
- Match: +1 → **--match** 1
- Mismatch: -3 → **--mismatch** -3
- Gap extend: -1 → **--gapextend** -1
- Gap open: -6 → **--gapopen** -6
- Seed gap (query): 5000 → **--max_seed_gap_query** 5000
- Seed gap (ref): 5000 → **--max_seed_gap_ref** 5000
- Restriction sites:
 - Upstream: →
--seed_pattern_upstream AAGCTT **--seed_upstream_score** 4
 - Downstream: →
--seed_pattern_downstream GATC **--seed_downstream_score** 4
- Inter-chromosome: enabled → **--interchr**

7.10. DNA Homology Search

Usage:

```
VAT nucl homology null -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 15 → **-S** 15
- Window: 3 → **-w** 3
- Match: +5 → **--match** 5
- Mismatch: -4 → **--mismatch** -4
- Gap extend: -3 → **--gapextend** -3
- Gap open: -5 → **--gapopen** -5

7.11. Whole Genome Alignment

Usage:

```
VAT nucl wga null -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Seed length: 18 → **-S** 18
- Window: 10 → **-w** 10
- Match: +1 → **--match** 1
- Mismatch: -5 → **--mismatch** -5
- Gap extend: -2 → **--gapextend** -2
- Gap open: -5 → **--gapopen** -5
- X-drop: 24 → **--xdrop** 24
- X-drop gapped: 24 → **-X** 24

7.12. Protein Homology Search

7.12.1. Fast

Usage:

```
VAT protein homology fast -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Pattern number: 3 → **-N** 3
- Gap extend: -1 → **--gapextend** -1
- Gap open: -11 → **--gapopen** -11

7.12.2. Sensitive

Usage:

```
VAT protein homology sensitive -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Pattern number: 10 → **-N** 10
- Gap extend: -1 → **--gapextend** -1
- Gap open: -11 → **--gapopen** -11

7.13. Translated Homology Search

7.13.1. Fast

Usage:

```
VAT blastx homology fast -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Pattern number: 3 → **-N 3**
- Gap extend: -1 → **--gapextend -1**
- Gap open: -11 → **--gapopen -11**

7.13.2. Sensitive

Usage:

```
VAT blastx homology sensitive -d <ref.fa> -q <reads.fq> -o <out.sam> -f sam
```

Parameters:

- Pattern number: 10 → **-N 10**
- Gap extend: -1 → **--gapextend -1**
- Gap open: -11 → **--gapopen -11**